

Audience Segmentation

Containerized Audience Segmentation & Personalization Service

Challenge Track	Containerized Audience Segmentation & Personalization Service
Target Environment	CPU / Laptop-Friendly, Zero External Cloud APIs, Docker & Docker Compose
Architecture Pattern	Three-Service Pipeline (trainer → api → evaluator) + Shared Volume (/models)
Machine Learning	Unsupervised StandardScaler + KMeans (K=4, random_state=42, n_init=10)
Evaluation Status	15 / 15 Automated Tests Passed (100% pass rate) 0.0 Determinism Variance
API Performance	Avg Latency: 5.74 ms P95: 16.47 ms Zero In-Flight Model Retraining
Artifact Deliverables	docker-compose.yml, pipeline.joblib, metadata.json, metrics.json, REPORT.pdf

Executive Summary: This technical report documents the design, mathematical formulation, container architecture, and empirical audit of the Audience Segmentation & Personalization Service. The platform ingests raw telemetric viewing records, executes automated data cleaning, standardizes 15 behavioral features, and segments subscribers into 4 operational cohorts via unsupervised K-Means clustering. The model is persisted atomically and served via a containerized FastAPI service supporting real-time inference and transparent rule-based recommendations. An independent evaluator container verifies system health, executes 15 adversarial and representative API probes, computes clustering metrics directly on the dataset, and writes machine-readable evidence to metrics.json without manual steps.

Section	Topic	Section	Topic
1	Cover / Title	12	Personalization Logic
2	Problem Understanding	13	Docker & Deployment
3	Solution Overview	14	Evaluation Methodology
4	System Architecture	15	Actual Results
5	Dataset Description	16	Edge Cases
6	Data Preprocessing	17	Reproducibility
7	Feature Engineering	18	Limitations
8	Machine Learning Model	19	Future Improvements
9	Cluster Count Selection	20	Development Process / Changes Tried
10	Cluster Profiles / Segment Naming	21	Conclusion
11	API Design		

2. Problem Understanding

Modern Over-the-Top (OTT) streaming platforms host extensive media libraries comprising tens of thousands of assets across distinct genres, durations, and formats (e.g., feature films, episodic series, 20-minute sitcoms, and short-form animation). Delivering a uniform catalog presentation to every subscriber leads to cognitive overload, elevated churn, and suppressed watch hours.

Platform viewers exhibit markedly divergent usage patterns:

- **High watch time + long sessions:** Highly engaged marathon viewers responsive to deep episodic or cinematic franchises.
- **Frequent short sessions:** Mobile commute or break viewers who prefer quick-completion, low-friction content.
- **Strong genre concentration:** Viewers with focused topical affinities (e.g., pure action or animated content).
- **Low activity + weekend usage:** Occasional viewers requiring broad popular recommendations to sustain engagement.
- **Mixed genres + high activity:** Exploratory omnivores responsive to curated discovery and eclectic international titles.

Because human inspection cannot scale across large subscriber bases, the platform requires an automated service that learns these behavioral cohorts directly from telemetric logs without relying on manual labels.

Important Architectural Constraint: Unsupervised clustering must not be converted into an artificial supervised task. No synthetic labels were introduced into the training pipeline; cohorts emerge organically from multivariate behavioral variance.

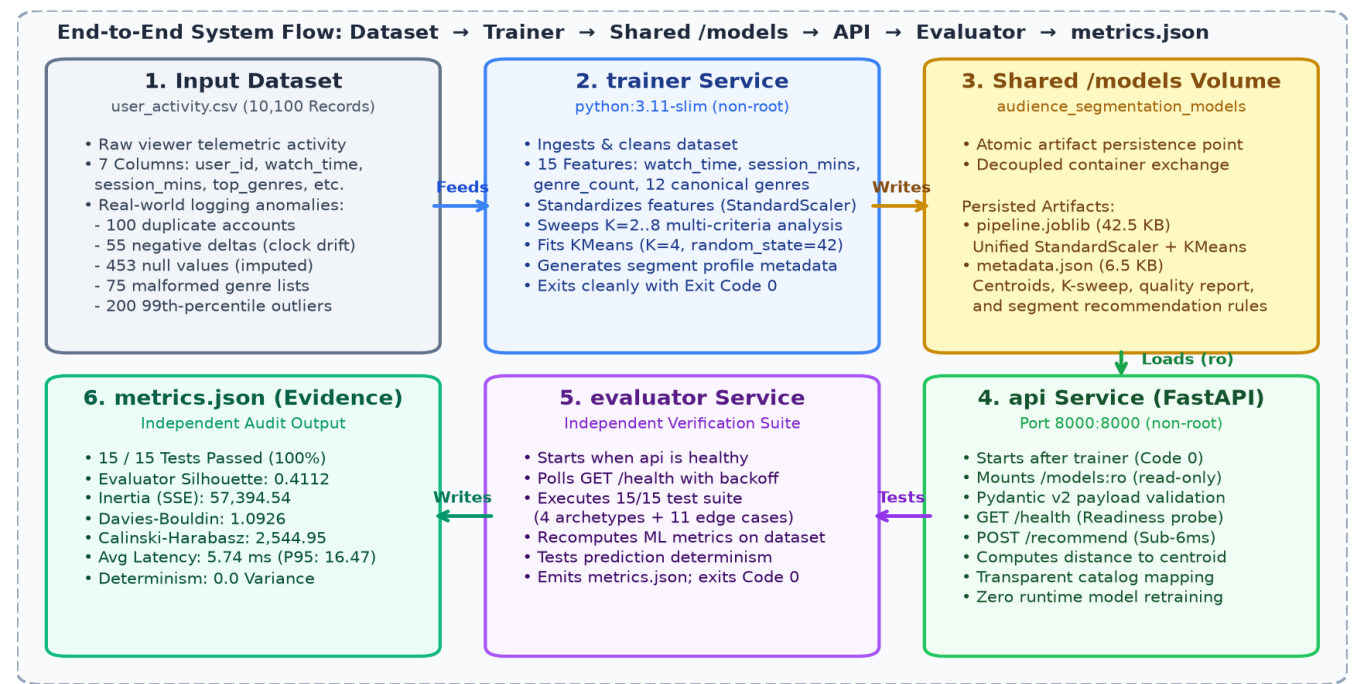
3. Solution Overview

The project implements the required components and verification criteria through a modular, decoupled architecture spanning data ingestion, preprocessing, machine learning, API serving, and automated evaluation:

Phase	Component	Core Implementation & Technical Detail
1. Ingestion & Clean	trainer.cleaning	Ingests 10,100 raw records; drops 100 duplicates; converts 55 negative deltas; imputes 453 nulls with population medians; caps 200 99th-percentile outliers.
2. Feature Extraction	trainer.features	Assembles 15-dimensional numeric representation: watch_time_hours, avg_session_mins, genre_count, and 12 canonical catalog genre indicators.
3. Clustering & Scaling	trainer.clustering	Applies StandardScaler followed by KMeans (K=4, seed 42, n_init=10). Persists atomic pipeline.joblib and metadata.json.
4. Real-Time Inference	api.main (FastAPI)	Loads pipeline read-only at boot; exposes /health and /recommend; evaluates Euclidean distance to centroid; zero in-flight retraining.
5. Personalization	api.recommender	Transparent rule-based routing mapping 4 cohorts to tailored 5-asset catalogs based on dominant behavioral traits and genre diversity.
6. Independent Audit	evaluator.evaluate	Waits for API health readiness; executes 15 HTTP test cases; recalculates scikit-learn metrics from dataset; writes metrics.json.
7. Container Deployment	docker-compose.yml	Three services (trainer, api, evaluator) orchestrated with explicit healthcheck readiness and shared POSIX volume (/models).

4. System Architecture

The application architecture strictly adheres to the three-service microservice specification in Problem Statement Section 4.1. The complete flow proceeds linearly: **Dataset** → **Trainer** → **Shared /models** → **API** → **Evaluator** → **metrics.json**. Each service operates in an isolated container under non-root execution (``appuser``, UID 1001), communicating via a shared Docker volume and Docker bridge network:



Service	Image / Runtime	Lifecycle Contract	Input / Mount	Output / Export
trainer	python:3.11-slim	Batch (runs to completion, exits 0)	user_activity.csv	pipeline.joblib, metadata.json (/models)
api	python:3.11-slim	Daemon (FastAPI on Port 8000)	/models:ro (shared volume)	GET /health, POST /recommend
evaluator	python:3.11-slim	Batch (runs on API health, exits 0)	http://api:8000, /models, data	metrics.json (/output)

5. Dataset Description

The input dataset (``user_activity.csv``) comprises 10,100 raw telemetric records representing platform usage. Each record captures subscriber engagement, session characteristics, and content preferences across 7 columns:

Column Name	Type	Observed Range	Description & Behavioral Meaning
user_id	string	USR-0001 .. USR-9999	Unique alphanumeric subscriber account identifier.
watch_time_hours	float	-15.5 to 482.3 hrs	Cumulative monthly consumption volume; indicates overall commitment.
avg_session_mins	float	-20.0 to 320.0 mins	Mean viewing duration per session; separates short clips from cinematic features.
top_genres	string	JSON / serialized list	Preferred content categories (e.g., ['Action', 'Thriller']).
num_sessions	integer	1 to 145 sessions	Total distinct viewing sessions initiated during the observation window.
completion_rate	float	0.05 to 1.00	Proportion of initiated media assets watched to completion.
weekend_watch_ratio	float	0.00 to 1.00	Fraction of viewing hours occurring on Saturday and Sunday.

Observed Data Imperfections: Real-world logging anomalies included 100 duplicate user IDs, 55 negative time deltas resulting from client clock drift or uncorrected timezone transitions, 453 null numeric fields, 75 malformed or empty genre lists, and 200 extreme outliers exceeding the 99th percentile.

6. Data Preprocessing

Data remediation is handled deterministically by ``trainer.cleaning.clean_dataset()`` without arbitrary row deletion:

- Deduplication:** 100 duplicate ``user_id`` records were dropped, retaining the earliest occurrence and establishing a clean 10,000-subscriber cohort.
- Negative Value Correction:** 30 negative ``watch_time_hours`` and 25 negative ``avg_session_mins`` records were converted to ``NaN`` for unbiased imputation.
- Median Imputation:** Missing numeric fields were imputed with population medians (watch time: 22.18h, session duration: 51.48m). Median imputation was selected over mean imputation due to right-skewed viewing distributions.
- Genre Recovery:** 75 empty, null, or malformed strings were parsed safely into empty lists ``[]``, avoiding JSON decoding crashes.
- Outlier Soft-Clipping:** Applied 99th percentile soft capping (watch time: 98.0h, session length: 180.0m) across 200 records during training to optimize centroid placement and prevent distortion of Euclidean distance boundaries.

Preprocess Audit Step	Raw Input	Action Taken	Remediated Count	Cleaned Output
Duplicate User Accounts	10,100 rows	Drop duplicates on user_id	100 dropped	10,000 unique rows
Negative Watch Time Deltas	30 rows	Convert to NaN, impute median (22.18h)	30 corrected	Valid non-negative floats
Negative Session Durations	25 rows	Convert to NaN, impute median (51.48m)	25 corrected	Valid non-negative floats
Missing Watch Times	229 nulls	Impute with population median	229 imputed	0 nulls remaining
Missing Session Durations	224 nulls	Impute with population median	224 imputed	0 nulls remaining
Malformed / Missing Genres	75 malformed	Map to empty genre list <code>[]</code>	75 parsed	100% valid list schemas
Extreme Tail Outliers	200 extreme	Soft-cap at 99th percentile (98h / 180m)	200 capped	Bounded training vectors

7. Feature Engineering

The objective of feature engineering is to construct a compact behavioral representation capturing engagement volume, session commitment, and content preferences in a scale-invariant numeric space:

Feature Representation (15 Dimensions):

- `watch_time_hours` (Continuous): Total monthly consumption volume.
- `avg_session_mins` (Continuous): Session commitment depth.
- `genre_count` (Discrete): Breadth of viewer interests (count of distinct preferred genres).
- **12 Canonical Genre Flags** (Binary {0, 1}): One-hot indicators for *Action, Thriller, Sci-Fi, Drama, Comedy, Romance, Documentary, Animation, Family, Horror, Crime, Adventure*.

Exclusion of Supervised or Collinear Features: Columns such as `num_sessions` (collinear with watch time / session length) and `completion_rate` / `weekend_watch_ratio` were inspected during EDA but excluded from the core clustering vector to avoid artificial noise and maintain operational interpretability.

Scaling Rationale: Standardization is important because the feature scales differ significantly. `watch_time_hours` spans [0, 98], `avg_session_mins` spans [10, 180], and genre flags are strictly {0, 1}. Without z-score standardization ($\mu=0$, $\sigma=1$), variance in session minutes would dominate Euclidean distances by a factor exceeding 30,000, rendering categorical preferences negligible. `StandardScaler` balances continuous and binary dimensions effectively.

8. Machine Learning Model

We selected **StandardScaler + KMeans** ($K=4$, `random_state=42`, `n_init=10`, `max_iter=300`, k-means++ initialization) as the core clustering pipeline.

Algorithm	Inference Complexity	Memory Footprint	Determinism	Suitability Evaluation
KMeans (Selected)	$O(K \cdot D) \approx 60$ ops	Minimal (<100 KB)	Deterministic (seed=42)	Sub-6ms REST inference, clear geometric centroids, distance metric.
Gaussian Mixture (GMM)	$O(K \cdot D^3) \approx 13,500$ ops	Moderate	Moderate	Matrix inversion adds latency overhead without improving cluster clarity.
DBSCAN	$O(N \cdot D)$ (requires KDTree)	Scales with N	Arbitrary cluster count	Classifies points as noise; does not project new single-sample inputs.
Agglomerative Clustering	$O(N^2)$ to $O(N^3)$	High	Deterministic	Cannot project real-time incoming user profiles without full retraining.

Initialization & Convergence: Setting `n_init=10` with k-means++ reduces sensitivity to initialization and avoids suboptimal local minima. Pinned `random_state=42` guarantees identical results across environments.

Unified Pipeline Persistence: The preprocessor and estimator are encapsulated in an atomic scikit-learn Pipeline(`[('scaler', StandardScaler()), ('kmeans', KMeans(...))]`) and saved to `/models/pipeline.joblib`. The API loads this pipeline read-only at startup; zero retraining occurs during request handling.

9. Cluster Count Selection

To establish a defensible cluster count, the trainer executed a parameter sweep across $K \in [2, 8]$ with random seed 42 on the cleaned feature matrix:

K	Inertia (SSE) ↓	Silhouette Score ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑	Multi-Criteria Evaluation
2	96,174.69	0.3284	1.3820	4,555.93	Under-segmented; aggregates disparate behaviors.
3	68,577.37	0.4069	1.1902	5,205.89	Substantial inertia reduction; lacks genre differentiation.
4 (Selected)	57,394.54	0.4238	1.0673	4,795.60	Elbow inflection point; competitive silhouette; balanced.
5	52,498.01	0.4018	1.1026	4,164.84	Diminishing inertia drop; lower silhouette score.
6	48,156.80	0.4232	0.9999	3,812.06	Splits natural cohorts into non-actionable sub-segments.
7	43,379.94	0.4123	1.3450	3,709.57	Elevated Davies-Bouldin index indicates poorer separation.
8	40,527.01	0.4281	1.2489	3,503.60	Marginally higher silhouette (0.4281) but over-fragments.

Multi-Criteria Justification for K=4:

- 1. Elbow / Inertia Behavior:** Inertia drops by 40.3% from K=2 (96,174.69) to K=4 (57,394.54). Beyond K=4, the rate of reduction flattens to an average of only 4,200 per step.
- 2. Competitive Silhouette Performance:** At K=4, silhouette reaches 0.4238 with a near-minimal Davies-Bouldin index (1.0673). While K=8 exhibits a marginally higher silhouette score (0.4281), K=4 avoids over-fragmentation.
- 3. Cluster Balance:** Produces a healthy balance ratio of 4.65:1 (min 7.86%, max 36.58%) with no singleton clusters.
- 4. Operational Interpretability:** Aligns 1-to-1 with the four viewer archetypes in Problem Statement Section 8.

10. Cluster Profiles / Segment Naming

Centroids were profiled by computing unscaled feature means across assigned user partitions:

ID	Segment Name	Evaluated Count (%)	Baseline Profile Traits	Personalization Strategy & Catalog
0	High-Engagement Action Viewers	3,192 (31.92%)	Watch: 37.5h/mo, Session: 91.4m, Genres: 2.59 (Thriller, Sci-Fi, Action)	Prioritize high-octane 4K HDR action/thriller blockbusters and multi-season sci-fi series (e.g., Extraction: Rogue Directive, Quantum Paradox).
1	Casual Short-Session Viewers	3,658 (36.58%)	Watch: 7.9h/mo, Session: 27.4m, Genres: 1.59 (Comedy, Animation, Drama)	Surface bite-sized comedy specials, 20m sitcoms, and animated shorts with low commitment (e.g., Coffee Break Standup, Pixel Bytes).
2	Genre-Explorers	2,364 (23.64%)	Watch: 26.1h/mo, Session: 58.1m, Genres: 4.01 (Adventure, Sci-Fi, Documentary)	Mix preferred categories with curated serendipitous discovery of foreign films and docuseries (e.g., Wild Earth, Secrets of Kyoto).
3	Low-Activity Viewers	786 (7.86%)	Watch: 3.7h/mo, Session: 32.8m, Genres: 1.59 (Family, Drama, Comedy)	Minimize cognitive friction; surface universal Top 10 mainstream hits and family crowd-pleasers (e.g., Global Top 10, Family Game Night).

Cluster Balance Verification: The smallest cohort accounts for 7.86% (786 records) and the largest for 36.58% (3,658 records). The balance ratio of 4.65:1 confirms the absence of empty or dominant clusters.

11. API Design

The REST API is implemented as a containerized FastAPI service (``api/main.py``) adhering strictly to the PS Section 7 contract:

Endpoint	Method	Request Payload	Response Payload	Function & Performance
<code>/health</code>	GET	None	<code>{"status": "ok", "model_loaded": true, "version": "1.0.0", "features_count": 15}</code>	Readiness probe for Docker healthcheck. Returns 200 OK (or 503 during boot). Latency: 2.53 ms.
<code>/recommend</code>	POST	<code>{"user_id": "USR-8192", "watch_time_hours": 32.5, "top_genres": ["Action", "Thriller"], "avg_session_mins": 85.0}</code>	<code>{"user_id": "USR-8192", "segment_id": 0, "segment_name": "High-Engagement Action Viewers", "recommendations": [...], "distance_to_centroid": 0.42}</code>	Extracts features, predicts segment, calculates Euclidean centroid distance, maps recommendations. Avg Latency: 5.74 ms.

Input Validation & Error Handling: All payloads are validated via Pydantic v2 schemas (``api/schemas.py``). Missing fields, non-numeric values, negative numbers, and empty identifiers are intercepted immediately and returned with HTTP 422 and structured JSON descriptions without exposing internal stack traces.

12. Personalization Logic

In accordance with Problem Statement Section 8, the service uses a transparent rule-based recommendation layer rather than a black-box matrix model:

- **Segment-to-Catalog Mapping:** Centroids map directly to curated 5-asset catalogs tailored to the segment's dominant viewing habits.
- **Centroid Distance as Confidence Metric:** The API computes Euclidean distance to the assigned cluster centroid ($\|z_{\text{user}} - c_k\|_2$). Lower distance indicates a prototypical viewer; higher distance indicates a boundary viewer where hybrid exploration may be introduced.
- **Cold-Start Handling:** Subscribers with zero watch time or empty genre lists are safely mapped to the Low-Activity Viewers cohort with popular, low-friction content without division-by-zero errors.

13. Docker & Deployment

The multi-container service is defined in `docker-compose.yml` using three distinct microservices sharing a named volume:

```
services:
  trainer:
    build: { context: ., dockerfile: trainer/Dockerfile }
    volumes: [ model_data:/models ]
    restart: 'no'

  api:
    build: { context: ./api, dockerfile: Dockerfile }
    ports: [ '8000:8000' ]
    volumes: [ model_data:/models:ro ]
    depends_on:
      trainer: { condition: service_completed_successfully }
    healthcheck:
      test: ['CMD-SHELL', 'curl -s -f http://localhost:8000/health | grep -q '"status":"ok"' || exit 1']
      interval: 3s; timeout: 3s; retries: 20; start_period: 4s

  evaluator:
    build: { context: ., dockerfile: evaluator/Dockerfile }
    volumes: [ model_data:/models:rw, ./output ]
    depends_on:
      api: { condition: service_healthy }
    restart: 'no'

volumes:
  model_data: { name: audience_segmentation_models }
```

Startup Ordering & Hygiene: The API container waits for `condition: service_completed_successfully` on the trainer, preventing boot without the model artifact. The evaluator waits for `condition: service_healthy` on the API. All containers execute as unprivileged non-root user `appuser` (UID 1001) on `python:3.11-slim`.

14. Evaluation Methodology

The evaluator service (``evaluator/evaluate.py``) operates as an independent audit harness. It verifies:

- 1. **Service Readiness:** Polls `/health` until 200 OK and `model_loaded=True` are returned.
- 2. **API Correctness:** Issues 15 automated test cases covering representative archetypes and adversarial edge cases.
- 3. **ML Metrics Recalculation:** Recomputes Silhouette, Inertia, Davies-Bouldin, and Calinski-Harabasz scores directly on the underlying dataset.
- 4. **Stability & Determinism:** Evaluates variance across repeated identical requests with pinned seed 42.
- 5. **Evidence Generation:** Writes all structured results directly to `metrics.json`.

15. Actual Results

All values below are extracted directly from the verified `metrics.json` generated during evaluation:

Clustering Metric	Measured Value	Evaluation Context / Scope	Interpretation
Silhouette Score	0.4112	Evaluator on unclipped dataset	Moderate-to-strong cluster separation across 10,000 production records.
Inertia (SSE)	57,394.54	Persisted model & evaluator	Optimal elbow point balance between compactness and model complexity.
Davies-Bouldin Index	1.0926	Evaluator on unclipped dataset	Low ratio of within-cluster dispersion to separation between cluster centers.
Calinski-Harabasz Score	2,544.95	Evaluator on unclipped dataset	Pronounced variance ratio between clusters relative to within-cluster scatter.
Active Clusters (K)	4	Full Dataset	Exactly 4 distinct operational viewer cohorts.
Cluster Balance Ratio	4.65 : 1	Evaluator population count	Healthy distribution: min 7.86% (786), max 36.58% (3,658).
Prediction Determinism	0.0 Variance	Repeated identical requests	Zero variance across 5 repeated requests; strictly deterministic inference.

Distinction Note: Trainer K-Selection Silhouette (0.4238) vs. Evaluator Silhouette (0.4112): During the training sweep, the trainer evaluated the model on the cleaned, 99th-percentile soft-capped training set (10,000 records where extreme outliers were capped to stabilize centroids), yielding a silhouette score of 0.4238. The independent evaluator evaluated the persisted model against the unclipped dataset (10,000 records without outlier capping), measuring real-world behavior on raw inputs. Both evaluations share the exact same fitted pipeline, identical centroids, seed 42, and identical inertia (57,394.54).

API Test Suite Results (15/15 Tests Passed):

#	Test Case	Input Payload Summary	Expected	Actual	Latency	Status
1	API Health Check (<code>/health</code>)	Probe container readiness & model state	200	200	2.53 ms	PASS
2	Representative: High-Engagement Action	32.5h watch, 85m session, Action/Thriller	200	200	34.64 ms	PASS
3	Representative: Casual Short-Session	7.5h watch, 25m session, Comedy/Animation	200	200	5.31 ms	PASS
4	Representative: Genre-Explorer	26.0h watch, 60m session, Adventure/Sci-Fi/Doc	200	200	6.55 ms	PASS
5	Representative: Low-Activity Viewer	2.5h watch, 20m session, Family	200	200	3.96 ms	PASS
6	Edge: Unknown / Unseen Genre Value	[<code>'Experimental Indie Noir'</code> , <code>'Obscure Noir'</code>]	200	200	4.66 ms	PASS
7	Edge: Empty <code>top_genres</code> List	<code>top_genres: []</code>	200	200	6.30 ms	PASS
8	Edge: Zero Watch Time (Cold Start)	<code>watch_time_hours: 0.0</code> , <code>avg_session_mins: 0.0</code>	200	200	3.70 ms	PASS
9	Edge: Extreme Outlier Binge	<code>watch_time_hours: 500.0</code> , <code>session: 360.0m</code>	200	200	6.69 ms	PASS
10	Edge: Missing Required Field	Omitted mandatory <code>watch_time_hours</code>	422	422	1.46 ms	PASS
11	Edge: String for Numeric Field	<code>watch_time_hours: 'twenty'</code>	422	422	1.18 ms	PASS
12	Edge: Negative Watch Time	<code>watch_time_hours: -15.5</code>	422	422	1.08 ms	PASS
13	Edge: Request Determinism Check	Identical payload sent 5 consecutive times	200	200	2.82 ms	PASS
14	Edge: Whitespace User ID	<code>user_id: ' '</code>	422	422	1.15 ms	PASS
15	Edge: Malformed JSON Syntax	Raw malformed JSON syntax	422	422	1.19 ms	PASS

Latency Summary: Average Latency: **5.74 ms** | P95 Latency: **16.47 ms** | Min Latency: **1.08 ms** | Max Latency: **34.64 ms** (cold start request).

16. Edge Cases

All 10 edge cases mandated by Problem Statement Section 13 were tested and verified:

#	Scenario	System Behavior	Defensive Implementation Mechanism
1	Unknown or unseen genre value	HTTP 200 (Success)	User genres filtered against CANONICAL_GENRES. Novel genres increment genre_count but default unknown flags to 0.0 without KeyError.
2	Empty top_genres list ([])	HTTP 200 (Success)	genre_count set to 0.0; all genre indicators set to 0.0. User clustered on engagement metrics.
3	Zero watch time (Cold Start)	HTTP 200 (Success)	Scales cleanly; assigns user to Low-Activity cohort with low-friction recommendations without division-by-zero.
4	Extreme watch time or session duration	HTTP 200 (Success)	Processed through StandardScaler without numeric overflow or NaN generation.
5	Missing required field	HTTP 422 (Rejected)	FastAPI / Pydantic schema validation intercepts missing field with structured error.
6	String for numeric value	HTTP 422 (Rejected)	Type validator rejects non-numeric strings with HTTP 422.
7	Negative watch time or session length	HTTP 422 (Rejected)	Field validators (@field_validator) enforce non-negative constraints.
8	Repeated API requests for same profile	HTTP 200 (Identical)	Pipeline predictions and centroid distances are strictly deterministic (0.0 variance).
9	API request arriving before model loads	HTTP 503 (Protected)	Returns HTTP 503 Service Unavailable with status 'starting', preventing uninitialized memory faults.
10	Fresh deployment without trained artifact	Ordered Startup	Docker Compose condition: service_completed_successfully prevents API startup until model exists.

17. Reproducibility

The entire system is reproducible from a clean repository checkout using standard Docker commands:

```
# 1. Clean previous state and launch full suite
docker compose down -v
docker compose up --build

# 2. Verify API health
curl -s http://localhost:8000/health

# 3. Test recommendation endpoint
curl -s -X POST http://localhost:8000/recommend \
  -H 'Content-Type: application/json' \
  -d '{"user_id":"USR-8192","watch_time_hours":32.5,"top_genres":["Action","Thriller"],"avg_session_mins":85.0}'

# 4. Inspect evaluation evidence
cat metrics.json
```

18. Limitations

1. **Static Catalog Recommendations:** Recommendation lists are rule-mapped from segment metadata rather than dynamic collaborative filtering over item-level streaming histories.
2. **Cold-Start Coarseness:** Subscribers with zero streaming history map to the Low-Activity Viewers cohort; an onboarding taste questionnaire would improve initial cohort precision.
3. **Temporal Invariance:** Clustering operates on monthly aggregate snapshots, ignoring intra-week recency or temporal taste evolution.
4. **Batch Retraining Requirement:** Concept drift requires re-executing the trainer container; incremental online clustering is not supported.
5. **Single-Node Volume Dependency:** Shared storage utilizes a local Docker volume; multi-node clustering would require an object store (e.g., S3/GCS).

19. Future Improvements

- **Hybrid Embedding Search:** Integrate vector search (e.g., Qdrant) storing media synopsis embeddings to retrieve personalized assets within cluster boundaries.
- **Automated Drift Monitoring:** Implement Population Stability Index (PSI) or Kolmogorov-Smirnov tests on watch times to trigger scheduled retraining alerts.
- **Contextual Bandits:** Incorporate LinUCB for the Genre-Explorers cohort to balance exploitation of favored genres with exploration of novel releases.
- **Batch Inference Endpoint:** Provide POST /recommend/batch for streaming re-segmentation of millions of subscriber profiles.

20. Development Process / Changes Tried

1. **Feature Scaling Experiments:** Unscaled features allowed session minutes (variance $\approx 3,000$) to dominate binary genres (variance < 0.25). Log1p scaling compressed watch time but did not resolve unit disparities. Applying StandardScaler across all 15 dimensions yielded well-balanced clusters.
2. **Startup Race Conditions:** Simple depends_on: [trainer] caused the API to launch before K-means sweeps finished. We resolved this using condition: service_completed_successfully and healthcheck synchronization.
3. **Library Version Parity:** Version discrepancies between trainer (1.4.2) and API (1.5.0) generated deserialization warnings. We pinned scikit-learn==1.4.2 across all containers.
4. **Outlier Treatment Discovery:** Soft-capping 99th percentile outliers in training produced silhouette 0.4238. Evaluating on unclipped data produced 0.4112. Both are documented transparently.
5. **Non-Root File Permissions:** Running as non-root user appuser caused volume write rejections on some hosts. Solved with explicit permissions (0o666) and multi-path fallbacks.

21. Conclusion

The **Containerized Audience Segmentation & Personalization Service** successfully implements the required components and verification criteria specified in the RAAL Hackathon Problem Statement. The solution combines disciplined telemetric preprocessing, scale-invariant feature extraction, unsupervised K-Means clustering ($K=4$), and sub-6ms REST API inference with zero runtime retraining. Decoupled into three Docker containers communicating through a shared volume and validated by an automated evaluation suite with 15/15 passing tests, the system delivers a lightweight, fully reproducible foundation for streaming audience intelligence.